

손영진



✉ ods04139@gmail.com
☎ 010-3975-5037
🌐 <https://sonyoungjin.netlify.app/>
🐙 github.com/sonprogrammer

GRADUATED

한서대학교

- 항공소프트웨어 공학과

SKILLS

Front-end

- Javascript
- Typescript
- React
- Next.js
- Tailwind CSS
- React-Query
- Zustand
- Recoil

Back-end

- Node.js
- Express.js
- MongoDB

CERTIFICATES

- 정보처리기사
- GTQ 1급
- 토익 835점

PROFILE

저는 일상생활에 가치를 얹어주고 싶은 일을 하고 싶습니다.
다양한 경험과 전공을 통해 가능성을 발견하면서 프론트엔드 분야에 집중적인 관심을 가지게 되었고, 단순히 '보이는 것'을 만드는 것을 넘어 사용자의 행동과 경험에 직접적인 영향을 줄 수 있다는 점에서 큰 매력을 느꼈습니다.
국비지원 교육을 수료하며 부족했던 부분을 보완했고, 일상에서 느꼈던 불편함을 바탕으로 여러 개인 프로젝트를 통해 아이디어를 구현해왔습니다.
사용자 경험 최적화와 웹 퍼포먼스 개선에 집중하며 성장하겠습니다.

EDUCATION

엘리스

웹 개발자 트랙 6기

08/2023 - 12/2023

- React / TypeScript 기반 프론트엔드 개발 및 팀프로젝트 2건 완료

PROJECTS

Mungpass - 개인 프로젝트

애견카페 통합 O2O 서비스

01/2026 - Present

기술스택 : Next, TypeScript, Tailwind CSS, Supabase(Auth, Realtime, DB, Storage),

TanStack Query, Zustand

구조 : FSD(Feature-Sliced Design) 아키텍처 적용

주요 구현 및 성과

- O2O 핵심 로직 설계
 - QR 코드 기반 입장 시스템 구축
 - Supabase Realtime과 React Query를 활용하여 체크인 즉시 사장 유저의 화면에서 현재 이용 현황을 실시간 반영, 별도 새로고침 없이 운영 가능
- FSD 아키텍처 기반 최적화
 - 코드 간 의존성 방향을 단방향으로 강제 하고 모듈 간 결합도를 최소화하여 코드 복잡도 개선
 - 비즈니스 로직이 UI 계층에 종속되지 않게 하여 추후 새로운 기능 도입을 할 때 기존 모듈을 수정 없이 재사용 가능한 구조로 설계하여 회귀 테스트 비용 절감
- 백엔드 보안 및 자동화 설계
 - 데이터 동기화 및 생애주기 자동화
 - 가입시 프로필 자동생성 및 매장 심사 상태에 따른 매장 테이블 자동 업데이트 등 데이터 흐름 자동화 구현
 - RLS 기반 데이터 격리
 - RLS정책을 적용하여 로그인한 사용자만 자신의 데이터에 접근할 수 있도록 권한을 제어하고 데이터 무단 접근을 차단
- 데이터 기반 B2B 관리 대시보드 구축
 - Refine 프레임워크 기반 관리자 전용 인터페이스 및 Recharts 활용 실시간 통계 시각화 구현
- AI 기반 매출 인사이트 분석 시스템 구축
 - 지능형 리포트 자동 생성
 - Gemini AI 연동을 통해 사장님 매출 데이터 기반의 리포트 자동 생성 프로세스 구축
 - 데이터 캐싱을 통한 API 호출 최적화
 - AI 분석 값을 DB에 저장하여 사용자 일일 평균 조회 횟수(약 3회) 기준 불필요한 API 호출을 약 67% 절감하여 데이터 조회 성능 극대화
- Lighthouse 기반 웹 퍼포먼스 최적화 및 PWA 도입
 - Service Worker 기반 PWA 환경 구축 및 뷰포트 설정을 통해 iOS 사파리 인풋창 확대 버그 해결
 - next/image의 priority 및 next/dynamic 활용 등 리소스 우선순위 조정 전략을 통해 LCP를 2초 이내로 유지하며 시각적 안정성(CLS) 0.001 이하 달성

Fuely - 개인 프로젝트

AI기반 개인 맞춤 영양 관리 웹 서비스

기술스택: Next.js, TypeScript, Zustand, React Query, Tailwind CSS, Material UI, Recharts, Groq AI API, Lucide React

개요

• AI 기반 일일 영양소 권장량 분석 및 식단 관리 웹 서비스

주요 구현 사항

• AI 연동 및 데이터 구조화

- Groq AI API 연동 및 AI의 비정형 응답을 JSON 형태로 파싱하여 시스템 데이터와 동기화하는 파이프라인 구축

• 보안 중심 인증 설계

- Edge Runtime 최적화
 - Next.js 미들웨어의 Edge 환경 구동을 위해 jose 라이브러리 채택
- 이중 보안 구조 설계
 - Access Token과 Refresh Token(HttpOnly Cookie)을 결합한 이중 보안 구조 설계로 XSS 및 세션 탈취 위험 최소화
- 미들웨어 기반 라우트 보호
 - Next.js 미들웨어를 활용한 라우트 가드 구축으로 서버 사이드에서의 효율적인 인증 체크 및 비인가 접근 차단

• 네트워크 최적화

- Zustand와 React Query
 - 서버 데이터는 React Query로 캐싱 및 동기화로 불필요한 네트워크 요청 최소화
 - 전역 UI 상태는 Zustand로 분리하여 불필요한 리렌더링 방지
- 인터셉터 기반 인증 로직
 - 20여개의 API 파일에 파편화되어 있던 인증 로직을 Axios 인터셉터로 401 에러 및 토큰 재발급 로직 일원화로 중복 코드 70%이상 감축
 - 토큰 재발급 상태 관리와 대기 요청 큐를 활용한 동시성 제어를 구현하여 다중 API 요청 환경에서도 안정적인 인증 유지 및 불필요한 서버 부하 최소화

VC(Virtual Coin) - 개인 프로젝트

실시간 모의 코인 투자 서비스

기술스택: React, TypeScript, Node.js, MongoDB, React Query, Styled-components, Tailwind CSS, Chart.js, Upbit WebSocket

주요 구현 사항

• 하이브리드 시세 수신 전략

- 기존 WebSocket만으로 데이터를 수신하던 방식에서 REST API 기반의 데이터 우선 렌더링과 WebSocket 기반의 백그라운드 동기화를 결합한 하이브리드 전략 채택
- 초기 시세 로딩시간을 약 3.3초에서 0.7초로 단축하여 사용

• 실시간 데이터 제어

- Upbit WebSocket 연동 시세 제어 및 Chart.js 가격 차트 시각화 구현

• 안정적인 보안 설계

- Node.js, MongoDB 기반 REST API 구축 및 미들웨어를 통한 사용자 권한 검증 검사

• 렌더링 성능 최적화

- WebSocket을 통한 실시간 시세 수신 시 lodash throttle 전략 적용하여 데이터 수신 주기를 제어하고 과도한 상태 업데이트를 방지함
- Recoil 상태를 atom 단위 분할을 통해 리렌더링 최소화

• 데이터 흐름 및 속도 개선

- Lighthouse Speed Index 및 FCP 기준 최적화 수행, 실시간 데이터 시각화의 반응성 강화
- 초기 측정 대비 TBT, CLS 지표를 유의미하게 단축하여 사용자 입력 반응성 및 레이아웃 안정성을 확보

BNTY(Be Next To You) - 개인 프로젝트

트레이너와 회원간 소통 플랫폼

기술스택: React, TypeScript, JavaScript, Node.js, MongoDB, WebSocket, Styled-components, Tailwind CSS, PWA

주요 구현 사항

• 경험 기반의 문제 해결 및 서비스 설계

- 트레이너 근무 당시 수기 출석 누락 및 잔여 수업 횟수 정산 관리의 불투명성과 개인정보 관리의 허점을 발견하여 서비스 기획부터 DB설계 진행

- QR코드 출석 체크 시스템을 도입하여 스캔과 동시에 서버로 데이터가 전송되어 수업 횟수 차감 및 출석 데이터가 자동으로 동기화되도록 구현
- **백엔드 아키텍처**
 - Node.js와 MongoDB 기반 MVC 패턴 REST API 설계 및 CRUD 기능 구현
- **실시간 소통을 위한 데이터 동기화**
 - WebSocket을 활용한 실시간 채팅 시스템을 구축하고, 편의를 위해 읽음/안읽음 상태 관리 시스템 개발
- **사용자 인터페이스 및 경험 최적화**
 - Styled-components
 - 컴포넌트 내부 로직에 따른 동적 스타일링(Props 기반 제어) 및 재사용성이 중요한 UI 요소 처리 최적화
 - Tailwind CSS
 - 유틸리티 방식을 통해 레이아웃 및 스타일을 빠르게 구현하여 개발 속도 향상
 - 위 두 기술의 결합을 통해 유지 보수성과 개발 생산성을 동시에 확보
 - 자동 로그인 기능 및 Skeleton UI 적용으로 사용자 경험 개선
- **성능 지표 최적화**
 - React.lazy 및 Suspense를 활용한 코드 스플리팅으로 초기 번들 사이즈를 최적화하여 렌더링 속도 개선
 - Promise.all 기반 데이터 병렬 처리 구조로 리팩토링하여 네트워크 대기 시간 및 데이터 로딩 지연 최소화
 - LCP 향상
 - 기존 8.0초 -> 1.14초로 단축
- **렌더링 최적화**
 - 메모이제이션 혹은 통해 불필요한 리렌더링을 방지 및 TBT 단축

Mingle - 팀프로젝트

플레이리스트 공유 SNS 서비스

기술스택: React, TypeScript, Recoil, React Query, Tailwind CSS, Twin.macro, Axios, OpenWeatherMap API

주요 역할

- **클라이언트 및 서버 상태 관리 활용**
 - React Query의 캐싱 전략을 도입하여 플레이리스트 등 조회 빈도가 높은 데이터의 불필요한 네트워크 호출을 감소 시킴
 - Recoil을 통해 전역상태를 관리 하여 컴포넌트 간 의존도를 낮추고 데이터 상태 변화에 따라 UI가 즉각적으로 동기화되도록 설계하여 개발 생산성 확보
- **Axios 인터셉터 기반 인증 흐름 통합 및 최적화**
 - Axios 인터셉터를 활용하여 토큰 재발급 로직을 프로젝트에 안정적으로 통합
- **애자일 기반 소통 및 조율**
 - 팀원들과 유연하게 화면 흐름을 논의했으며, API 명세서나 기획이 변경되는 과정에서도 백엔드 팀원과 매일 스크럼 미팅을 통해 명세 불일치로 인한 연동 이슈를 주도적으로 소통하여 반복적인 코드 수정 없이 프로젝트 일정 내 완료